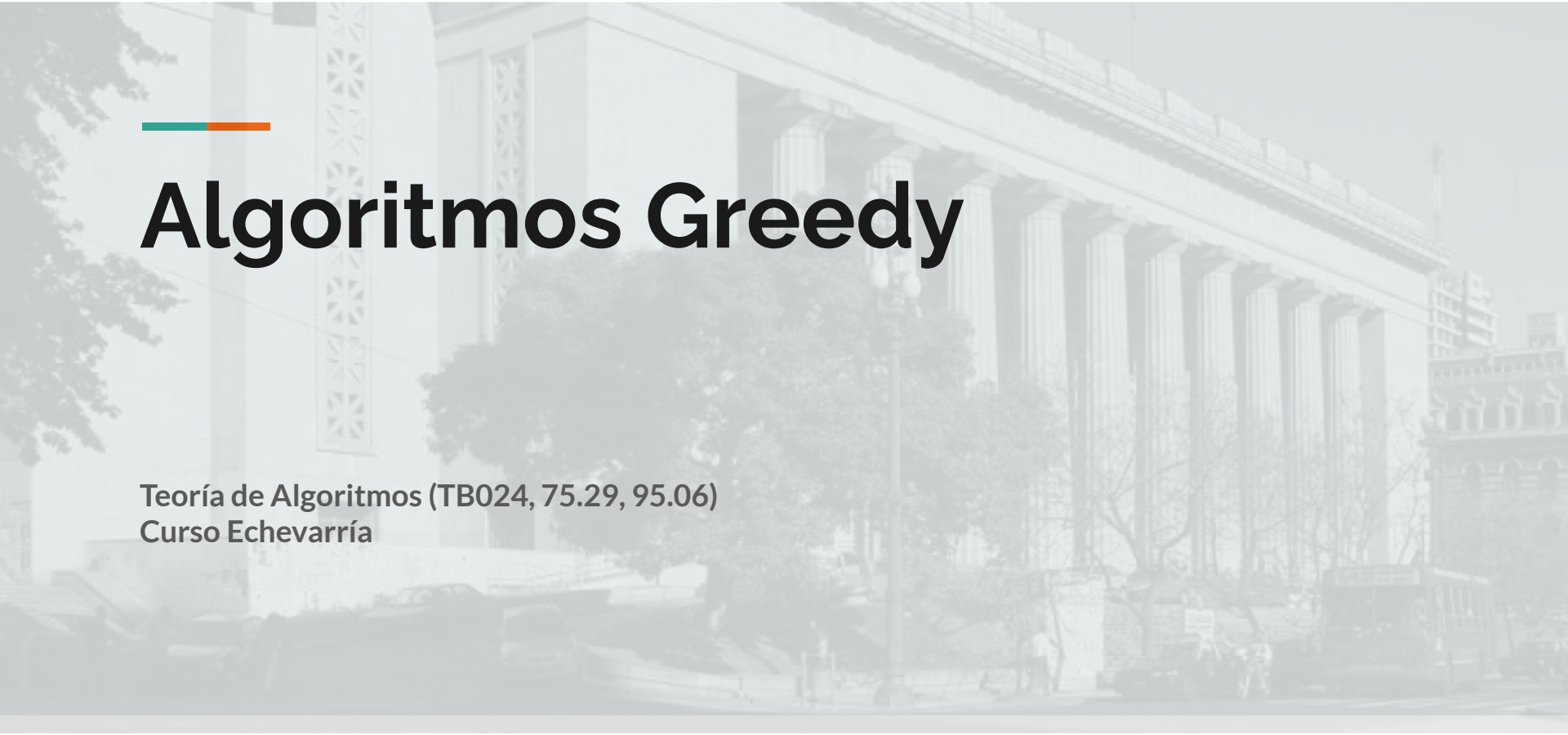
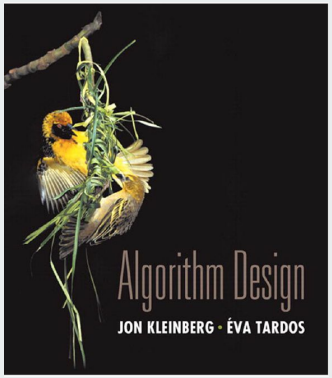


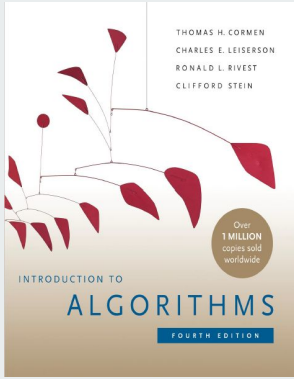
Algoritmos Greedy

Teoría de Algoritmos (TB024, 75.29, 95.06)
Curso Echevarría

Algoritmos Greedy



Cap 4



Cap 15



¿Qué son?

Algoritmos Greedy/Voraces/Golosos/Codiciosos

- Tratan de encontrar una solución óptima global a un problema dado
- Aplican una regla sencilla en cada paso: toman la decisión localmente óptima para el estado actual
- Iteran y vuelven a aplicar la regla para tomar la decisión localmente óptima del nuevo estado actual
- Pueden considerar estados o resultados previos pero no futuros

***Con suerte**, la aplicación iterativa de la regla nos lleva al óptimo global*

Revisitando Dijkstra

¿Para qué sirve el algoritmo de Dijkstra?

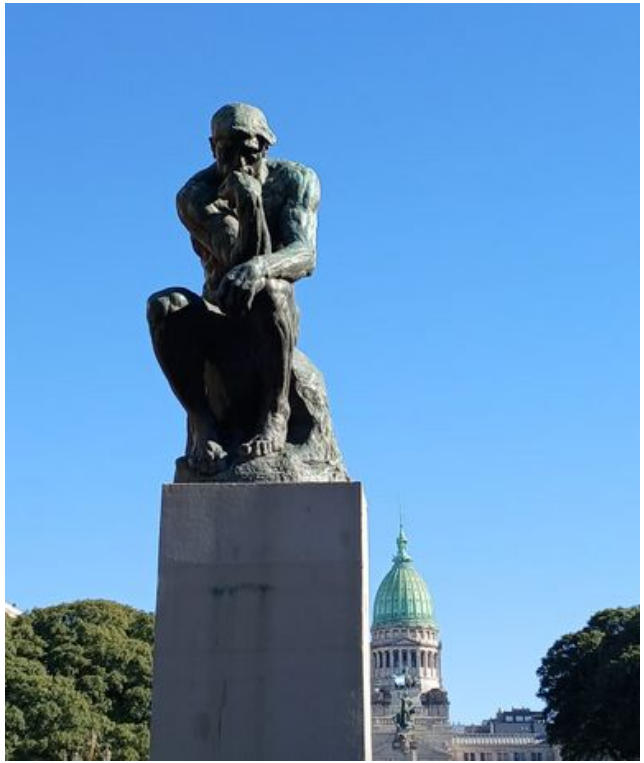
- Busca el camino mínimo desde un vértice hasta otro
- Condiciones:
  - Para grafos ponderados (con peso)
  - No debe tener pesos o ciclos con peso negativos

¿Cómo funciona el algoritmo de Dijkstra?

- Parte de un vértice
- Busca el siguiente vértice que haga mínimo el camino completo
- Se apoya en un heap para mejorar la búsqueda del siguiente vértice

Revisitando Dijkstra (cont.)

¿Es un algoritmo Greedy?



“El Pensador” - A. Rodin

Revisitando Prim

¿Para qué sirve el algoritmo de Prim?

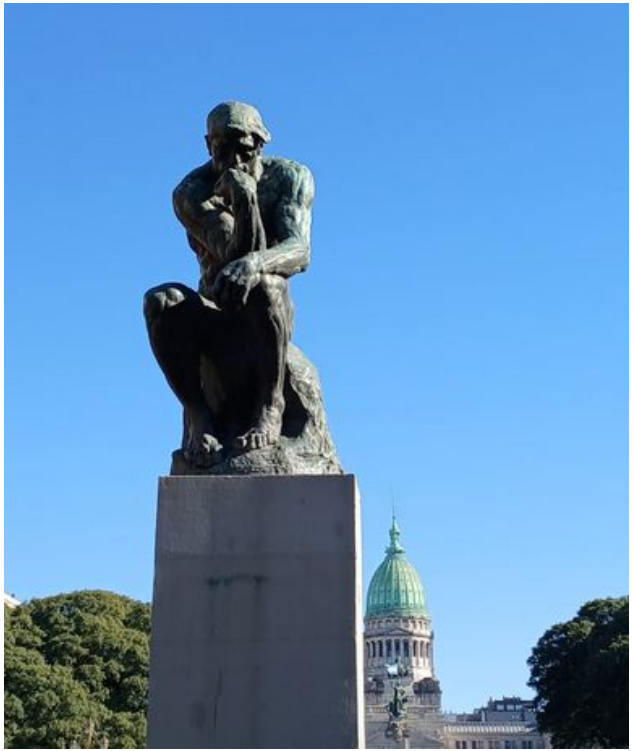
- Busca el árbol de tendido mínimo (MST) de un grafo
- Condiciones:
  - Para grafos no dirigidos ponderados (con peso)

¿Cómo funciona el algoritmo de Prim?

- Parte de un vértice arbitrario
- Busca la siguiente arista de menor peso posible
- Controla que no se formen ciclos

Revisitando Prim (cont.)

¿Es un algoritmo Greedy?



Revisitando Kruskal

¿Para qué sirve el algoritmo de Kruskal?

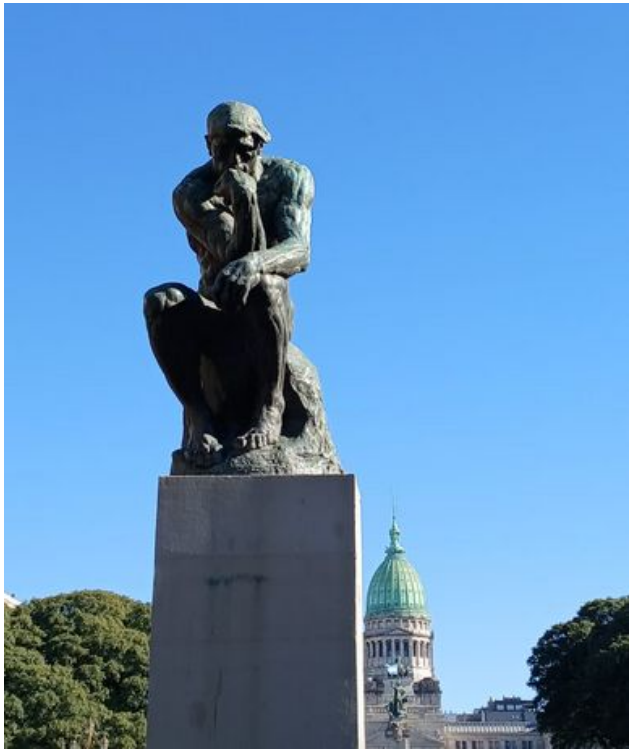
- También busca el árbol de tendido mínimo (MST) de un grafo
- Condiciones:
  - Para grafos no dirigidos ponderados (con peso)

¿Cómo funciona el algoritmo de Kruskal?

- Ordena las aristas por peso
- Incorpora la arista de peso mínimo
- Controla que no se formen ciclos con una estructura union-find

Revisitando Kruskal (cont.)

¿Es un algoritmo Greedy?



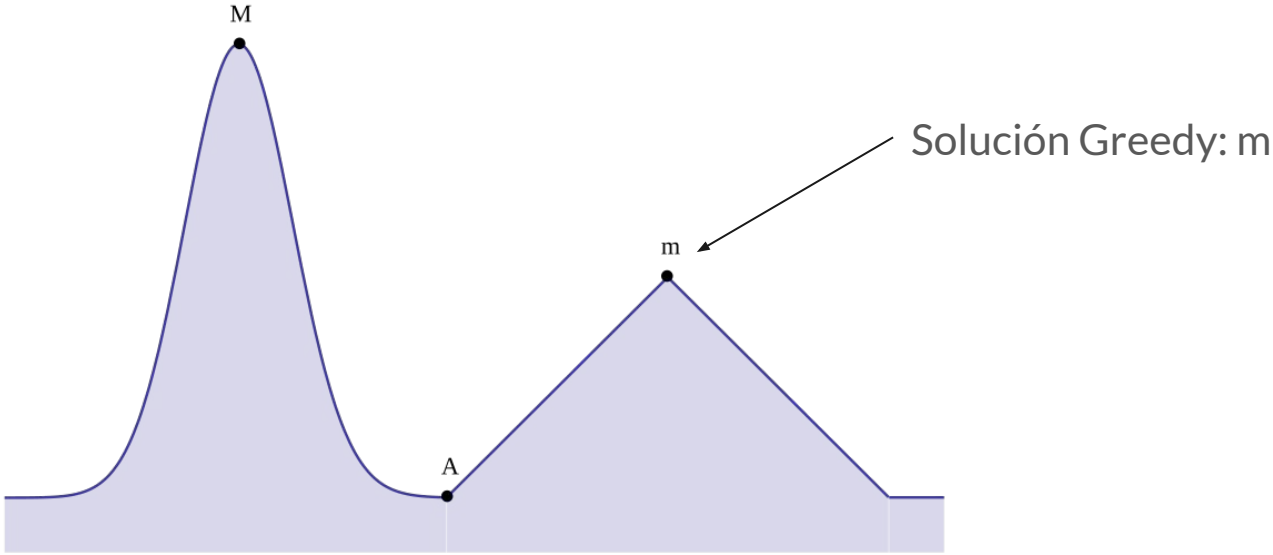
Pros y contras de los Algoritmos Greedy

- Suele ser difícil demostrar que un algoritmo greedy halle siempre el óptimo global
- Pueden fallar catastróficamente
- Son intuitivos, fáciles de entender
- Son de implementación sencilla
- Son rápidos
- Aunque no encuentren el óptimo global, se pueden aprovechar:
  - Como aproximación a la solución óptima
  - Como entrada a otro algoritmo más complejo

Ejemplo de solución desastrosa (1)

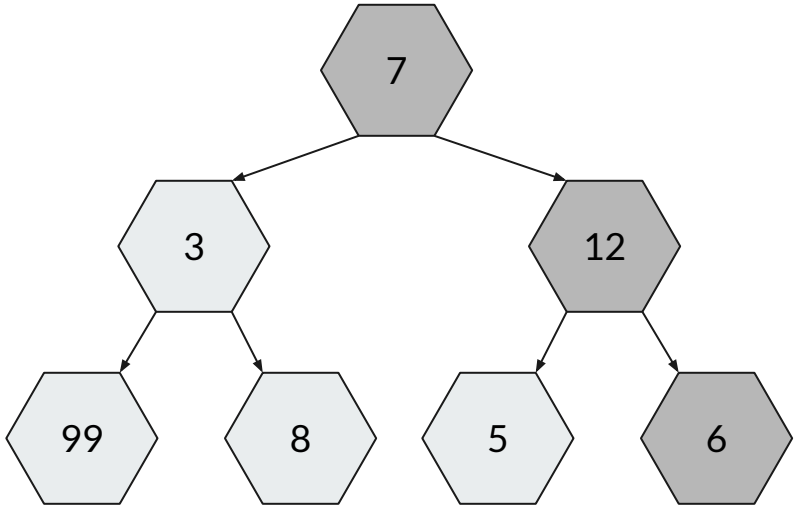
Buscar el máximo partiendo de A.

Regla Greedy: seguir el camino con la mayor pendiente



Ejemplo de solución desastrosa (2)

Calcular la suma máxima posible. Regla Greedy: sumar el próximo mayor término



Interval Scheduling

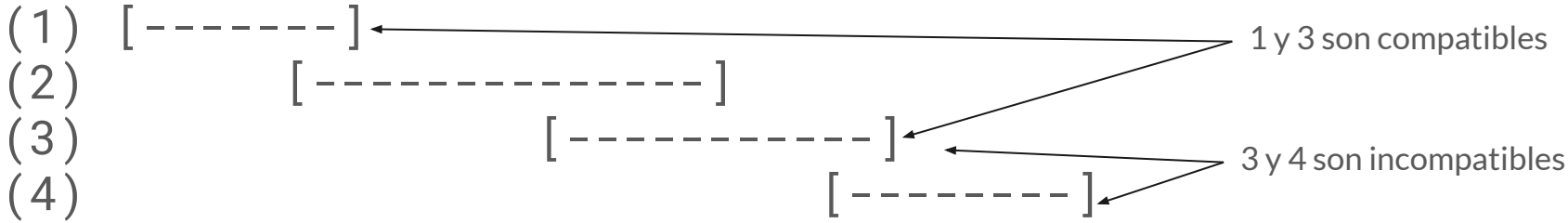
(Planificación de Intervalos)



Interval Scheduling: Definición

- Tenemos un conjunto de tareas $S=\{1,2,\ldots, n\}$
- La $i\text{-}ésima$ tarea empieza en el horario s_i y termina en el horario f_i (y no se puede modificar ni interrumpir)
- Definición: un subconjunto de tareas es compatible si no hay solapamiento entre ellas

Encontrar el subconjunto compatible más grande posible



Interval Scheduling (cont.)

Planteamos un algoritmo greedy para resolverlo:

1. Seleccionar una tarea i_1 siguiendo una regla sencilla
2. Rechazar todas las tareas que no sean compatibles
3. Seleccionar la segunda tarea i_2 para su consideración
4. Rechazar todas las tareas que no sean compatibles con i_2
5. Repetir hasta que no queden tareas

Interval Scheduling (cont.)

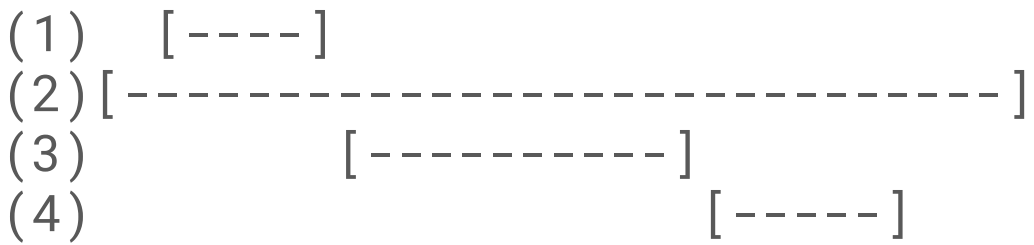
¿Cuál podría ser la regla sencilla a aplicar en nuestro algoritmo Greedy?

Interval Scheduling (cont.)

Opción 1: Tomar la próxima tarea que empiece lo antes posible

En este caso, nos aseguramos que el recurso esté ocupado lo más temprano posible

¿Qué pasa en este escenario?



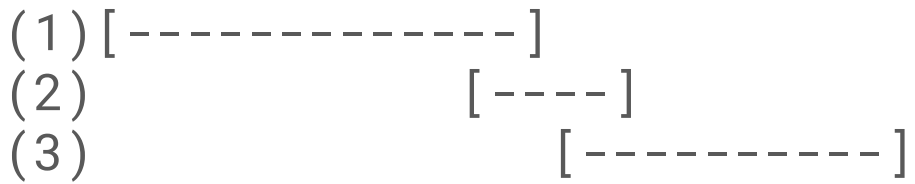
Tomar la tarea que primero empieza no es buena idea

Interval Scheduling (cont.)

Opción 2: Tomar la próxima tarea más corta posible

En este caso, nos aseguramos que el recurso tome muchas tareas cortas

¿Qué pasa en este escenario?

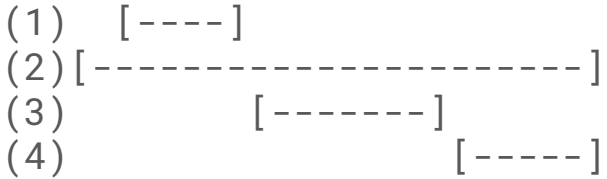


Tomar la tarea más corta tampoco es buena idea

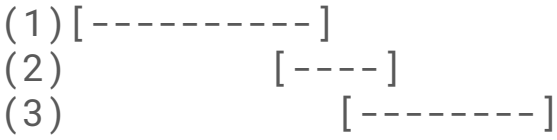
Interval Scheduling (cont.)

Opción 3: Tomar la próxima tarea que termine antes

En este caso, nos aseguramos que el recurso se libere lo antes posible, y le quede el mayor tiempo disponible restante



Escenario 1



Escenario 2

Tomar la tarea que termina antes es la forma óptima de calendarizar

Interval Scheduling: Pseudocódigo

Opción 3: Tomar la próxima tarea que termine antes

```
def scheduling(tareas)
  tareas_ordenadas=ordenar_tareas_por_fin(tareas)
  calendario=[]
  for tarea in tareas_ordenadas:
    if len(calendario)==0 or not incompat(calendario[-1], tarea)
      calendario.append(tarea)
  return calendario

def incompatibles(anterior, posterior)
  return posterior[INICIO] < anterior[FIN]
```

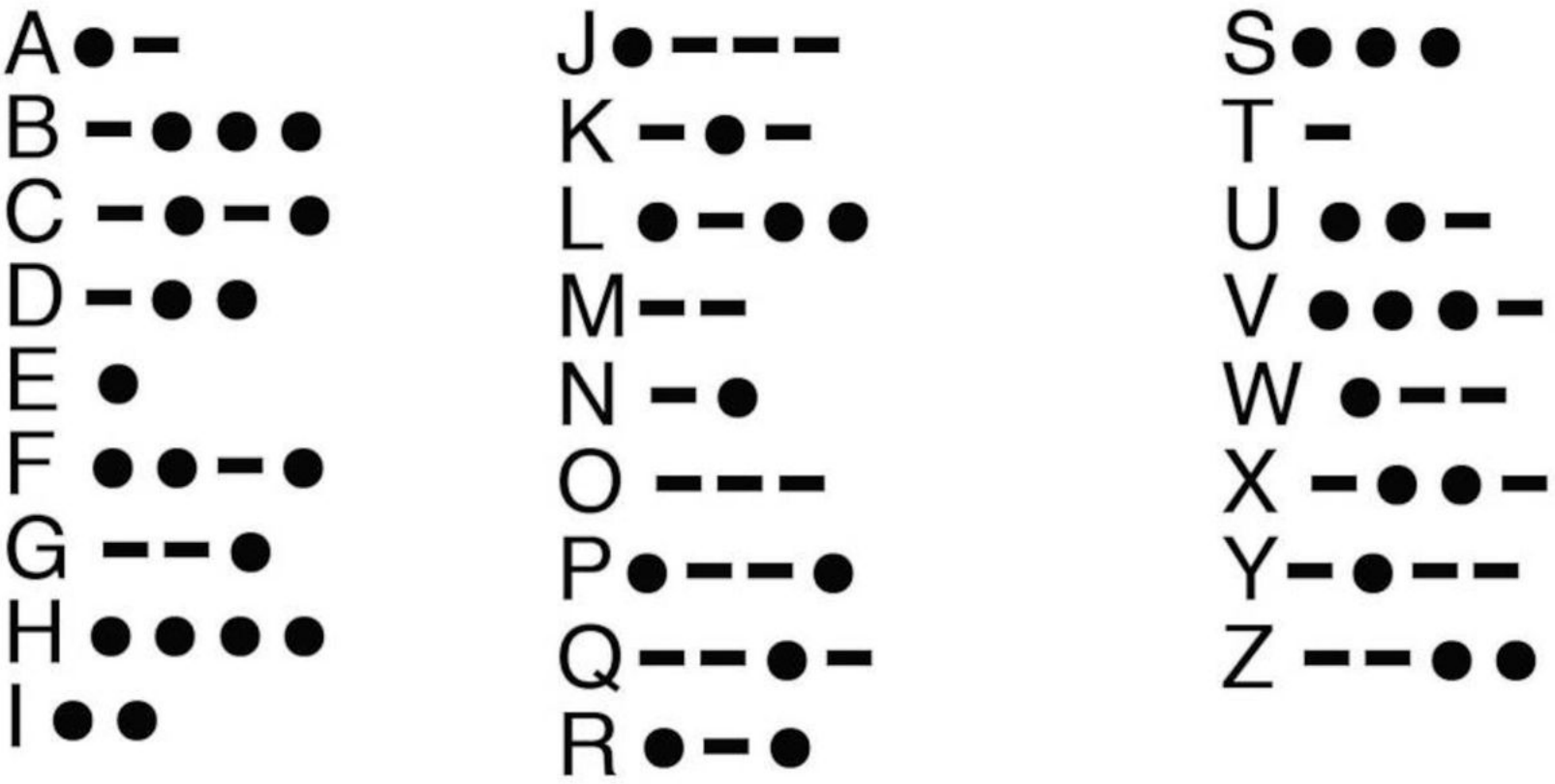
Códigos de Huffman



Codificación de Símbolos

- Normalmente usamos códigos del mismo tamaño para representar símbolos (p/ej.: 8 bits para caracteres ASCII)
- Si se quieren transmitir por un canal, eso no es eficiente
- ¿Qué tal si cada símbolo fuera codificado con códigos de diferente longitud?
  - Códigos más cortos para símbolos muy frecuentes
  - Códigos más largos para símbolos poco frecuentes
- Cuando hablamos de símbolos, hablamos de:
  - Caracteres
  - Bytes
  - etc.
- Una codificación de longitud variable es el código Morse

Código Morse



Códigos de Huffman

- Usados para compresión de datos sin pérdidas
- Permiten encontrar una codificación basada en la frecuencia de aparición de los símbolos
  - Reducir la longitud *promedio* de los códigos
- El objetivo es que los símbolos más frecuentes puedan ser codificados con menos bits que los menos frecuentes
- El algoritmo de generación usa un heap (de mínimo) para mantener los símbolos según sus frecuencias

Códigos de Huffman: Codificación

1. Determinar la frecuencia de los símbolos del mensaje
2. Cargar los símbolos y sus frecuencias en un heap
3. Mientras el heap no esté vacío:
    a. Tomar los próximos dos elementos del heap, “x”, “y”
    b. Construir un nuevo símbolo “xy” con frecuencia igual a la suma de las frecuencias de “x” y de “y”
    c. Agregar al heap el nuevo símbolo “xy” con su frecuencia
    d. Agregar al árbol T una nueva hoja “xy” con su frecuencia.
    e. Agregar debajo de la nueva hoja “xy” los nodos para “x” e “y” con sus frecuencias
4. Construir el código de Huffman asignando 0 y 1 a las aristas del árbol

Códigos de Huffman: Ejemplo

ABRACADABRA (88 bits)

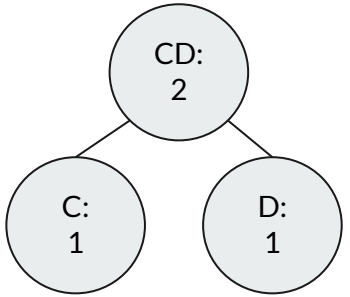
A	5
B	2
R	2
C	1
D	1

Tabla de Frecuencias

C	D	B	R	A
1	1	2	2	5

Heap Inicial

Códigos de Huffman: Ejemplo (cont.)

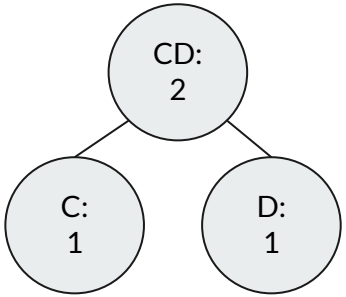


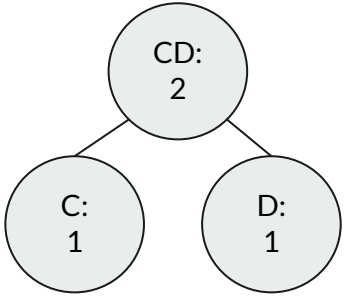
C	D	B	R	A
1	1	2	2	5

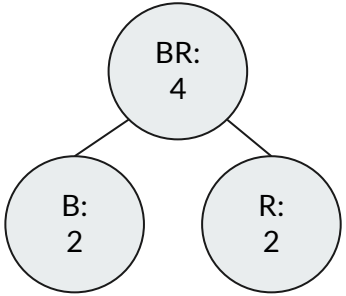


B	R	CD	A
2	2	2	5

Códigos de Huffman: Ejemplo (cont.)





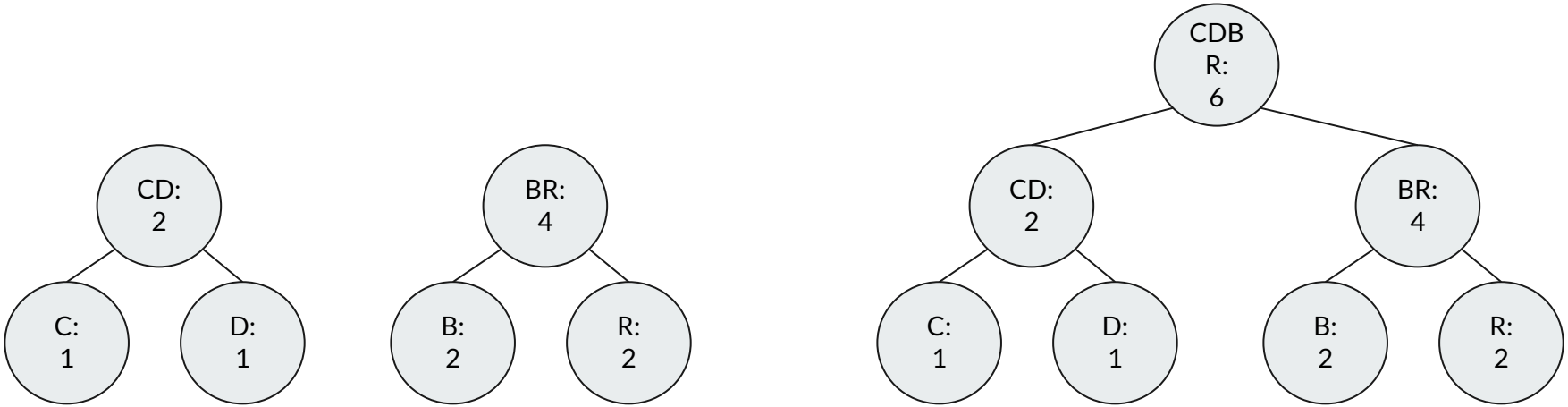


B	R	CD	A
2	2	2	5



CD	BR	A
2	4	5

Códigos de Huffman: Ejemplo (cont.)

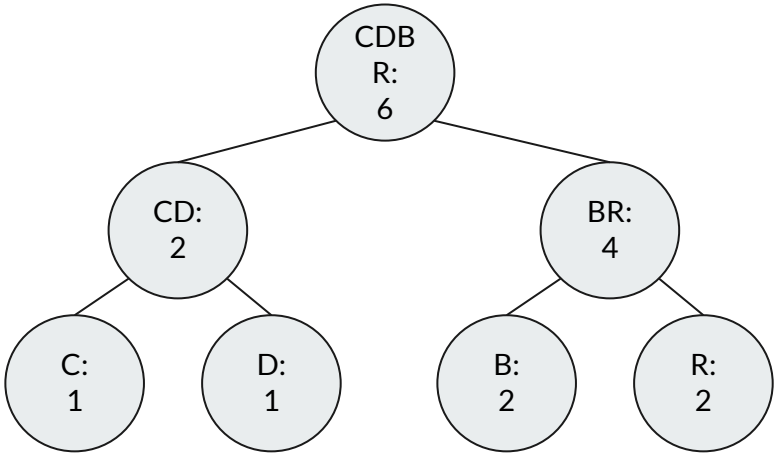


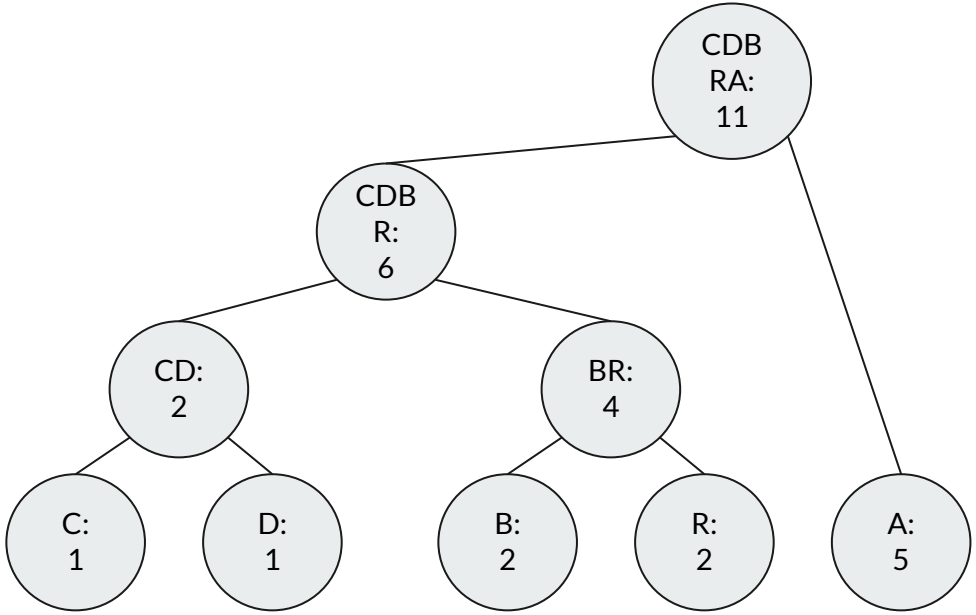
CD	BR	A
2	4	5



A	CDBR
5	6

Códigos de Huffman: Ejemplo (cont.)





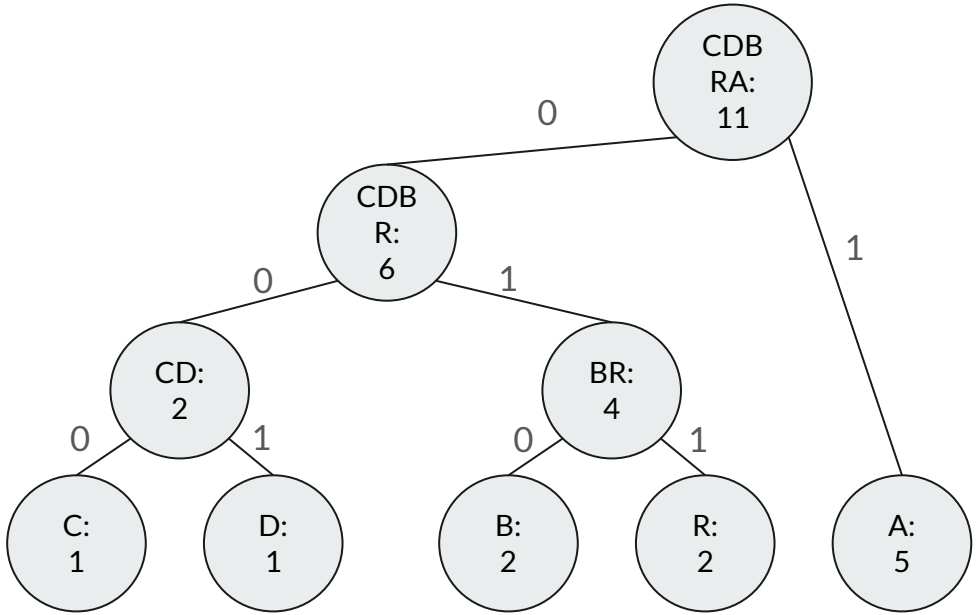
A	CDBR
5	6



CDBRA
11

Códigos de Huffman: Ejemplo (cont.)

A	5	1
B	2	010
R	2	011
C	1	000
D	1	001



ABRACADABRA:

1-010-011-1-000-1-001-1-010-011-1 (19 bits)

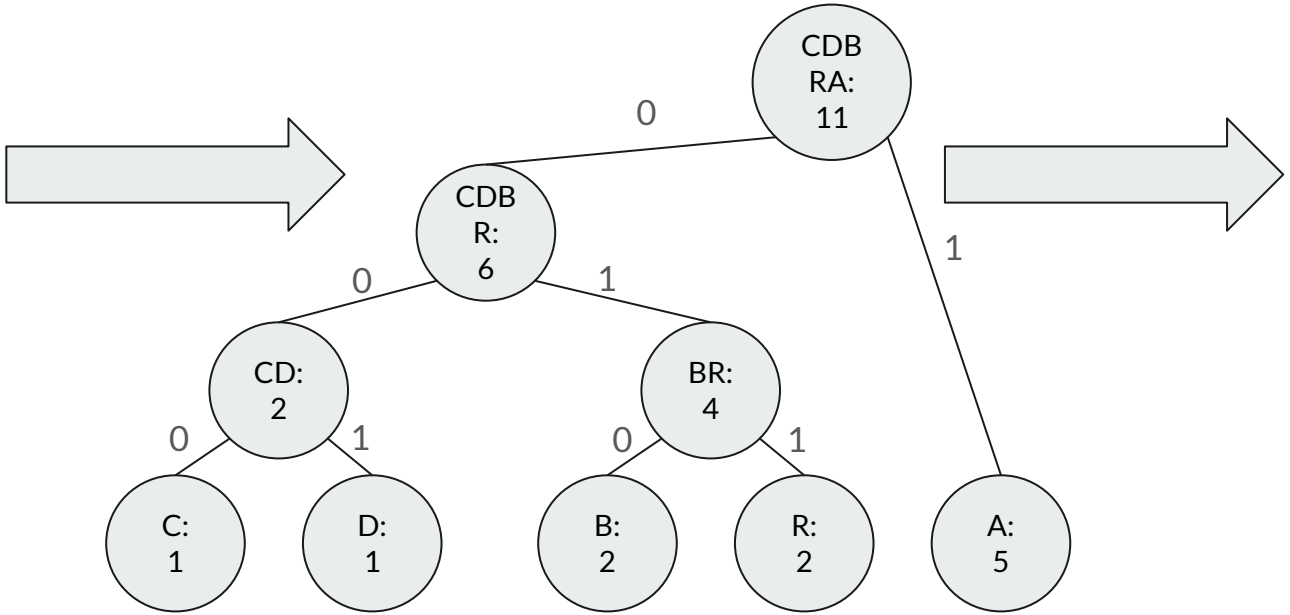
Códigos de Huffman: Decodificación

¿Cómo funciona la decodificación?

- El emisor envía el mensaje codificado junto con la tabla de frecuencias
- El receptor construye el código de Huffman y decodifica el mensaje

Códigos de Huffman: Ejemplo

A	5
B	2
R	2
C	1
D	1

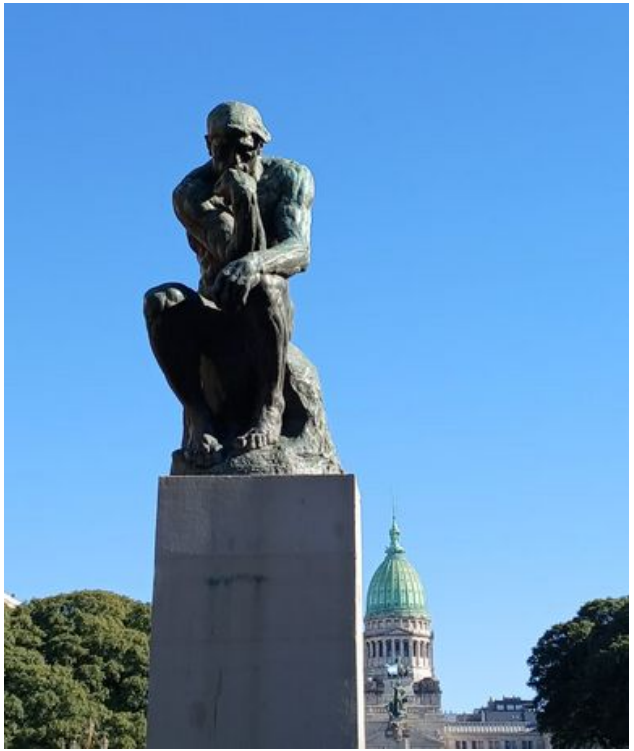


A	1
B	010
R	011
C	000
D	001

1-010-011-1-000-1-001-1-010-011-1 => ABRACADABRA

Códigos de Huffman (cont.)

¿Es un algoritmo Greedy?



Códigos de Huffman (cont.)

TL/DR: Sí, el algoritmo es greedy

¿Por qué es Greedy?

- Regla sencilla: tomar el próximo par de símbolos, que están priorizados de menor a mayor frecuencia de aparición
- Un símbolo con baja frecuencia terminará siendo representado con más bits, y viceversa

Reformulando la regla sencilla:

- Agregar un bit más a los dos símbolos que menor frecuencia acumulan

Códigos de Huffman: Pseudocódigo

```
def huffman(mensaje)
  frecuencias=calcular_frecuencias(mensaje)
  q=heap_crear()
  for simbolo in frecuencias
    q.encolar(Hoja(simbolo, frecuencias)
  while q.cantidad > 1
    t1=q.desencolar()
    t2=q.desencolar()
    q.encolar( Arbol(t1, t2, t.frecuencia + t2.frecuencia)
  return codificar(mensaje, q.desencolar())
```

Problema del Cambio de Monedas



Cambio de Monedas: Definición

Dado un sistema monetario S formado por n monedas $S=\{m_1, m_2, \ldots, m_n\}$, *encontrar la mínima cantidad de monedas necesarias para sumar cierto monto de dinero*

Cambio de Monedas (cont.)

Regla Greedy:

Usar la moneda de mayor denominación posible, todas las veces que se pueda

Cambio de Monedas (cont.)

Posible algoritmo:

1. Ordenar las monedas por denominación, de mayor a menor
2. Tomar la próxima denominación
3. Determinar cuántas veces entra la denominación en el monto a dar
4. Tomar el resto de la división entera
5. Repetir 2 hasta que el resto de la división entera sea cero

Cambio de Monedas: Ejemplo 1

Sistema Monetario S={1,2,5,10,50,100}

- Monto: 22. Solución: 2 monedas de 10, una moneda de 2
- Monto: 40. Solución: 4 monedas de 10
- Monto: 23. Solución: 2 monedas de 10, una de 2, una de 1

Cambio de Monedas: Ejemplo 2

Sistema Monetario S={5,10,20,25} (*)

- Monto: 40. Solución: 1 moneda de 25, una de 10, una de 5.
- O mejor, dos monedas de 20.

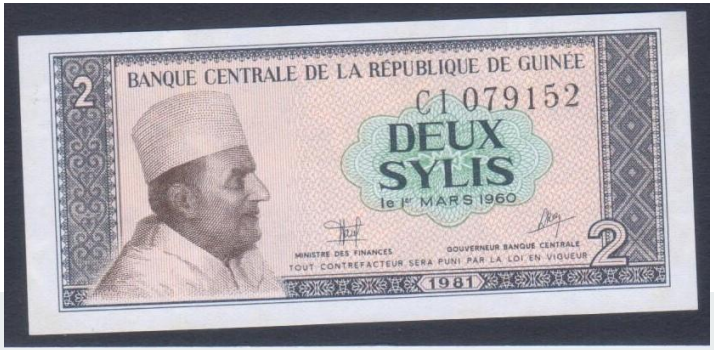
(*) Estas denominaciones funcionaron en India hasta 1968

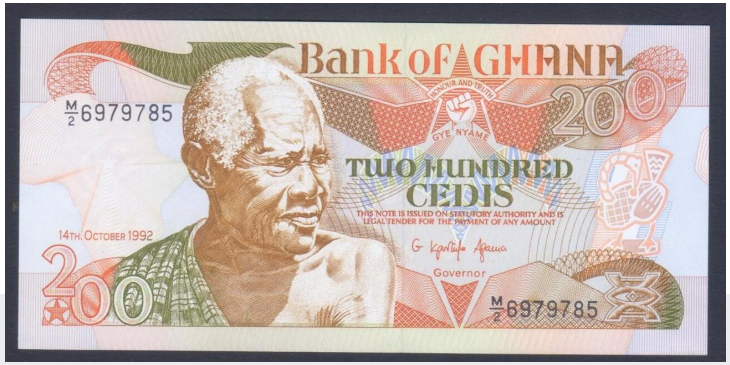
Cambio de Monedas: Conclusión

La optimalidad del algoritmo depende del sistema monetario









Compras con Inflación



Compras con inflación: Definición

- Se tiene un conjunto de n productos a comprar, cada uno con un precio $p_i > 1$
- Sólo se puede comprar un producto por día.
- Por cada día que pasa, los productos aumentan de precio en una tasa $t > 1$, de manera que luego de d días, el valor de p_i es $p_i t^d$

¿En qué orden habría que comprar los productos para que el costo total sea mínimo?

Compras con inflación (cont.)

Regla Greedy:

Comprar primero el más caro

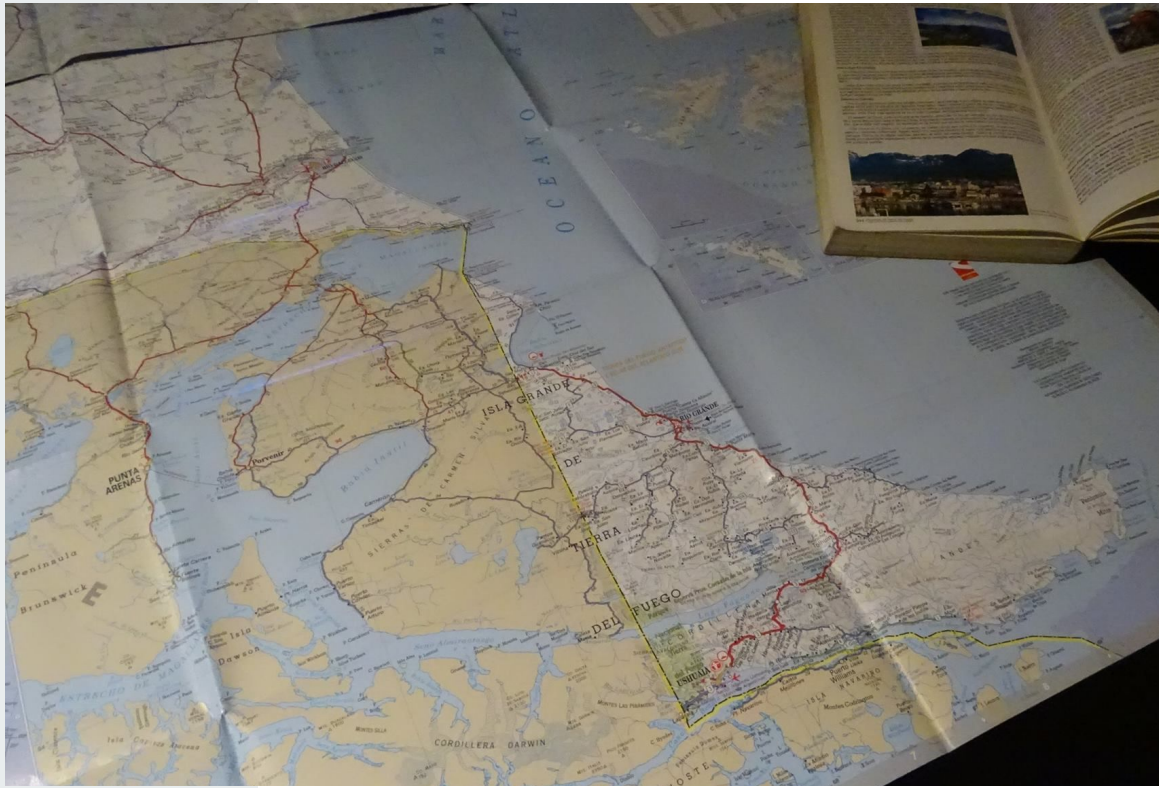
Compras con inflación (cont.)

Posible algoritmo:

1. Ordenar los productos por precio, de mayor a menor
2. Tomar el próximo producto
3. Acumular el costo
4. Remover el producto
5. Repetir desde 2 hasta que no queden más productos

Se puede demostrar que este algoritmo es óptimo

Carga de Combustible



Carga de Combustible: Definición

- Un camión debe recorrer una distancia r para ir desde un punto a otro
- Puede recorrer una distancia máxima k con su tanque lleno (autonomía)
- Las sucesivas estaciones de servicio $(e_{i-1}; e_i)$ a lo largo de la ruta están separadas por una distancia d_i.

Determinar en qué estaciones debe detenerse a cargar combustible para que la cantidad total de paradas sea la mínima posible

Carga de Combustible (cont.)

Regla Greedy:

Aplazar la carga lo más posible (*)

(*) Se debe conocer la ubicación de la próxima estación de servicio. No intenten esto en sus autos.

Carga de Combustible (cont.)

Posible algoritmo:

1. Inicializar el odómetro parcial $Op=o$
2. Tomar la distancia d a la siguiente estación
3. Si $d > k - Op$
   a. Cargar combustible
   b. Reiniciar el odómetro parcial $Op=o$
4. Si no
   a. $Op = Op + d$
5. Repetir hasta llegar a destino



¡Muchas gracias!